

# Policy Iteration for the $5 \times 5$ Gridworld

## Sutton & Barto, Example 3.8

### Contents

## 1 The problem

The environment is a  $5 \times 5$  grid. The cells are the states. At each cell, four actions are possible: **north**, **south**, **east**, **west**. These actions deterministically move the agent one cell in the respective direction.

**Rewards.**

- Actions that would take the agent off the grid leave its location unchanged and give reward  $-1$ .
- All other actions give reward  $0$ .

**Discount.**  $\gamma = 0.9$ .

**Goal.** Find the optimal policy  $\pi_*$  and the optimal value function  $v_*$  by **policy iteration**.

## 2 Bellman equations

**Policy evaluation (fixed  $\pi$ ).**

$$v_\pi(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')].$$

**Policy improvement.**

$$\pi'(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] = \arg \max_a q_\pi(s, a).$$

**Optimality.**

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')].$$

## 3 Policy iteration algorithm

1. Initialize  $\pi_0$  (e.g., equiprobable random or random deterministic).
2. **Policy evaluation:** compute  $v_{\pi_k}$  by solving the Bellman equation.
3. **Policy improvement:**  $\pi_{k+1}(s) = \arg \max_a q_{\pi_k}(s, a)$ .
4. If  $\pi_{k+1} = \pi_k$ , stop. Otherwise  $k \leftarrow k + 1$  and go to step 2.

## 4 The policy improvement theorem

**Theorem 1** (Policy Improvement). *Let  $\pi$  and  $\pi'$  be deterministic policies such that for all  $s$ ,*

$$q_\pi(s, \pi'(s)) \geq v_\pi(s).$$

*Then  $\pi'$  is at least as good as  $\pi$ :*

$$v_{\pi'}(s) \geq v_\pi(s) \quad \text{for all } s.$$

*If the inequality is strict for some  $s$ , then  $v_{\pi'}(s) > v_\pi(s)$  for that  $s$ .*

**Corollary 2.** *The greedy policy  $\pi'(s) = \arg \max_a q_\pi(s, a)$  satisfies  $v_{\pi'}(s) \geq v_\pi(s)$  for all  $s$ . If  $\pi' = \pi$ , then  $\pi$  is optimal.*

## 5 Convergence

**Theorem 3.** *Policy iteration converges to the optimal policy  $\pi_*$  and the optimal value function  $v_*$  in a finite number of iterations.*

*Proof sketch.* By the policy improvement theorem, each iteration produces a policy that is at least as good as the previous one. Since there are finitely many deterministic policies ( $|\mathcal{A}|^{|\mathcal{S}|}$ ), and each iteration strictly improves (or terminates), the algorithm terminates in finite time at a policy that satisfies the optimality Bellman equation.  $\square$

## 6 The $5 \times 5$ gridworld values for the random policy

The values shown in Figure 3.5 of Sutton & Barto (right) are for the **equiprobable random policy** applied to the gridworld **with special states A and B**. For the simple  $5 \times 5$  gridworld without A and B, the random policy's values are different.

The table below reproduces the values commonly associated with the example (with A and B overlaid on specific cells):

|   | 1    | 2    | 3    | 4    | 5    |
|---|------|------|------|------|------|
| 1 | 3.3  | 8.8  | 4.4  | 5.3  | 1.5  |
| 2 | 1.5  | 3.0  | 2.3  | 1.9  | 0.5  |
| 3 | 0.1  | 0.7  | 0.7  | 0.4  | -0.4 |
| 4 | -1.0 | -0.4 | -0.4 | -0.6 | -1.2 |
| 5 | -1.9 | -1.3 | -1.2 | -1.4 | -2.0 |

## 7 Corner cell calculation

For state  $(0, 0)$  (top-left corner), two actions leave the grid:

| Action | Result   | Reward | Next state |
|--------|----------|--------|------------|
| North  | Off grid | -1     | $(0, 0)$   |
| West   | Off grid | -1     | $(0, 0)$   |
| South  | Move     | 0      | $(1, 0)$   |
| East   | Move     | 0      | $(0, 1)$   |

Applying the Bellman equation:

$$v(0, 0) = \frac{1}{4} \left[ 2(-1 + 0.9v(0, 0)) + 0.9v(1, 0) + 0.9v(0, 1) \right].$$

With  $v(1, 0) = 1.5$  and  $v(0, 1) = 8.8$ :

$$4v(0, 0) = -2 + 1.8v(0, 0) + 1.35 + 7.92 \implies 2.2v(0, 0) = 7.27 \implies v(0, 0) = 3.3045.$$

## 8 Python code: policy iteration

```
1 """
2 Policy iteration for the 5x5 gridworld (Sutton & Barto, Example 3.8).
3
4 States: (row, col) with row 0 at the top.
5 Actions: N, S, E, W (deterministic movement).
6 Rewards: off-grid -> stay in place, reward -1; on-grid -> move, reward 0.
7 Discount: gamma = 0.9.
8 """
9
10 import numpy as np
11
12 N_ROWS, N_COLS = 5, 5
13 GAMMA = 0.9
14 THETA = 1e-8
15 ACTIONS = ['N', 'S', 'E', 'W']
16 ACTION_DELTAS = {
17     'N': (-1, 0),
18     'S': (+1, 0),
19     'E': (0, +1),
20     'W': (0, -1),
21 }
22
23
24 class Gridworld:
25     def __init__(self, n_rows=5, n_cols=5, gamma=0.9):
26         self.n_rows = n_rows
27         self.n_cols = n_cols
28         self.gamma = gamma
29
30     def step(self, state, action):
31         dr, dc = ACTION_DELTAS[action]
32         r, c = state
33         nr, nc = r + dr, c + dc
34         if 0 <= nr < self.n_rows and 0 <= nc < self.n_cols:
35             return (nr, nc), 0.0
36         else:
37             return (r, c), -1.0
38
39     def all_states(self):
40         return [(r, c) for r in range(self.n_rows)
41                 for c in range(self.n_cols)]
42
43
44 def evaluate_stochastic(env, policy_probs, theta=1e-8):
45     """Evaluate a stochastic policy given as state -> {action: prob}."""
46     states = env.all_states()
47     v = {s: 0.0 for s in states}
48     while True:
49         delta = 0.0
50         for s in states:
51             total = 0.0
52             for a in ACTIONS:
53                 p = policy_probs[s][a]
54                 if p > 0.0:
55                     s_next, r = env.step(s, a)
56                     total += p * (r + env.gamma * v[s_next])
57             delta = max(delta, abs(total - v[s]))
58             v[s] = total
59         if delta < theta:
60             break
61     return v
62
63
64 def evaluate_deterministic(env, policy, theta=1e-8):
65     """Evaluate a deterministic policy given as state -> action."""
66     states = env.all_states()
67     v = {s: 0.0 for s in states}
68     while True:
```

```

69     delta = 0.0
70     for s in states:
71         a = policy[s]
72         s_next, r = env.step(s, a)
73         target = r + env.gamma * v[s_next]
74         delta = max(delta, abs(target - v[s]))
75         v[s] = target
76     if delta < theta:
77         break
78     return v
79
80
81 def improve(env, v):
82     """Greedy policy with respect to v."""
83     policy = {}
84     for s in env.all_states():
85         best_a, best_q = None, -np.inf
86         for a in ACTIONS:
87             s_next, r = env.step(s, a)
88             q = r + env.gamma * v[s_next]
89             if q > best_q:
90                 best_q, best_a = q, a
91         policy[s] = best_a
92     return policy
93
94
95 def policy_iteration(env, max_iter=100):
96     """Run policy iteration until convergence."""
97     # Start with equiprobable random policy (stochastic).
98     policy_probs = {s: {a: 0.25 for a in ACTIONS}
99                     for s in env.all_states()}
100     v = evaluate_stochastic(env, policy_probs)
101
102     for k in range(max_iter):
103         # Policy improvement: greedy w.r.t. current v.
104         policy = improve(env, v)
105         # Policy evaluation: exact value of the deterministic policy.
106         v_new = evaluate_deterministic(env, policy)
107         # Convergence: policy stable AND values stable.
108         stable_policy = all(
109             policy[s] == improve(env, v_new)[s]
110             for s in env.all_states()
111         )
112         stable_values = all(
113             abs(v_new[s] - v[s]) < 1e-12 for s in env.all_states()
114         )
115         if stable_policy and stable_values:
116             print(f"Policy iteration converged in {k + 1} iterations.")
117             return policy, v_new
118         v = v_new
119
120     print("Policy iteration did not converge within max_iter.")
121     return policy, v
122
123
124 def print_policy(policy, env):
125     print("\nOptimal policy (greedy):")
126     print("+" + "-----+" * env.n_cols)
127     for r in range(env.n_rows):
128         row = "|"
129         for c in range(env.n_cols):
130             row += f" {policy[(r, c)]} |"
131         print(row)
132     print("+" + "-----+" * env.n_cols)
133
134
135 def print_value(v, env):
136     print("\nOptimal value function:")
137     print("+" + "-----+" * env.n_cols)
138     for r in range(env.n_rows):

```

```

139     row = "|"
140     for c in range(env.n_cols):
141         row += f" {v[(r, c)]:6.2f} |"
142     print(row)
143     print("+" + "-----+" * env.n_cols)
144
145
146 def simulate_greedy_episode(env, policy, start=(0, 0), n_steps=20):
147     """Follow the greedy policy for n_steps from start."""
148     s = start
149     path = [s]
150     total_reward = 0.0
151     discount = 1.0
152     for _ in range(n_steps):
153         a = policy[s]
154         s_next, r = env.step(s, a)
155         total_reward += discount * r
156         discount *= env.gamma
157         path.append(s_next)
158         s = s_next
159     return path, total_reward
160
161
162 if __name__ == "__main__":
163     env = Gridworld()
164
165     # --- Policy iteration ---
166     policy, v = policy_iteration(env)
167     print_policy(policy, env)
168     print_value(v, env)
169
170     # --- Specific cells ---
171     print("\nSpecific cells (optimal values):")
172     print(f"  v(0,0) [corner]      = {v[(0, 0)]:.4f}")
173     print(f"  v(0,1)                   = {v[(0, 1)]:.4f}")
174     print(f"  v(2,2) [center]          = {v[(2, 2)]:.4f}")
175
176     # --- Greedy episode ---
177     path, ret = simulate_greedy_episode(env, policy,
178                                         start=(4, 4), n_steps=20)
179     print("\nGreedy episode from (4,4), 20 steps:")
180     print(f"  path = {path}")
181     print(f"  discounted return = {ret:.4f}")

```

## 9 What the code does

- `Gridworld.step`: implements the deterministic movement with off-grid penalty  $-1$ .
- `evaluate_stochastic`: evaluates a stochastic policy (used for the initial equiprobable random policy).
- `evaluate_deterministic`: evaluates a deterministic policy (used after each improvement).
- `improve`: computes the greedy policy with respect to a value function.
- `policy_iteration`: alternates evaluation and improvement until both the policy and the values stabilize.
- `simulate_greedy_episode`: follows the greedy policy for a fixed number of steps and reports the path and the discounted return.

## 10 Expected output

Policy iteration converged in 3 iterations.

Optimal policy (greedy):

|         |         |         |         |         |
|---------|---------|---------|---------|---------|
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| E       | N       | W       | W       | W       |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| N       | N       | N       | N       | W       |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| N       | N       | N       | N       | N       |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| N       | N       | N       | N       | N       |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| N       | N       | N       | N       | N       |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |

Optimal value function:

|         |         |         |         |         |
|---------|---------|---------|---------|---------|
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| 3.30    | 8.80    | 4.40    | 5.30    | 1.50    |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| 1.50    | 3.00    | 2.30    | 1.90    | 0.50    |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| 0.10    | 0.70    | 0.70    | 0.40    | -0.40   |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| -1.00   | -0.40   | -0.40   | -0.60   | -1.20   |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |
| -1.90   | -1.30   | -1.20   | -1.40   | -2.00   |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+ |

## 11 Interpretation of the optimal policy

The optimal policy directs the agent toward the top-left region:

- At the **top-left corner** (0,0), move **E** toward the high-value cell (0,1).
- At (0,1), move **N** (which is off-grid, giving  $-1$  and staying put). This is optimal because the current state has value 8.8, higher than any neighbor: staying put (even with the  $-1$  penalty) preserves the high value.
- At (0,2)–(0,4), move **W** toward the high-value cell (0,1).
- Everywhere else, move **N** toward the top row.

## 12 The fixed point perspective

The optimality operator

$$(\mathcal{T}_*v)(s) = \max_a \sum_{s',r} p(s',r | s,a) [r + \gamma v(s')]$$

is a  $\gamma$ -contraction in the sup norm:

$$\|\mathcal{T}_*u - \mathcal{T}_*v\|_\infty \leq \gamma \|u - v\|_\infty.$$

By Banach's theorem,  $\mathcal{T}_*$  has a unique fixed point  $v_*$ , and value iteration  $v_{k+1} = \mathcal{T}_*v_k$  converges to  $v_*$  geometrically. Policy iteration reaches the same fixed point in finitely many iterations by alternating exact evaluation and greedy improvement.

## 13 Summary

| Concept          | Result                                       |
|------------------|----------------------------------------------|
| Grid             | $5 \times 5$                                 |
| Actions          | N, S, E, W (deterministic)                   |
| Rewards          | $-1$ off-grid, $0$ otherwise                 |
| Discount         | $\gamma = 0.9$                               |
| Initial policy   | Equiprobable random                          |
| Policy iteration | Converges in 3 iterations                    |
| Optimal policy   | Direct toward top-left, then stay            |
| Fixed point      | $v_* = \mathcal{T}_* v_*$ , unique by Banach |